

Fundamentals of Python Programming

Instructor: Samaher Alzahrani

Date: May 6, 2026

Class 1: Basics of Python

Topics Covered in This Course:

- Variables.
 - Strings.
 - Numbers.
 - Python Functions.
 - Conditional Statements.
 - Loops.
 - Python Collections.
 - Classes.
-

Variables

variables are names (or labels) that refers to an object stored in memory. Python objects can be of any type, including numbers, strings, lists, functions, and more.

```
In [2]: string = "Hello world"
        print(string)
```

Hello world

```
In [3]: string = "Python is fun!"
        print(string)
```

Python is fun!

Naming and Using Variables in Python

When naming a variable, ensure that its name is clear, readable, and descriptive of its purpose. Additionally, there are rules and guidelines you should follow when naming variables to avoid errors and prevent misleading or confusing names:

- A variable name can contain **letters, numbers, and underscores** (_).
- A variable name must **start with a letter or an underscore**, not a number such as `1_var`.

- **Spaces are not allowed** in variable names. Use underscores to separate words.
- Do **not use Python keywords or built-in function names** (like `print`, `list`, or `type`) as variable names.
- Use **short but descriptive names** that explain what the variable stores.
- Avoid confusing characters such as lowercase **l** and uppercase **O**, because they can be confused with the numbers 1 and 2.

```
In [ ]: var_1 = [1,2,3]
        Pi = 3.14
        ACCELERATION_DIRECTION = '-ve'
        print(var_1, "\n", Pi, "\n", ACCELERATION_DIRECTION)
```

```
In [ ]: 1_var = 1
```

Strings

A string is a sequence of characters, which can include letters, numbers, symbols, and even spaces.

```
In [ ]: print("Today is Tuesday")
        print('Today is Tuesday')
```

```
In [ ]: print('Afnan's book')
```

```
In [ ]: print("Afnan's book")
```

```
In [ ]: print('She said "Yes"')
        print("She said 'Yes'")
```

String's methods

In Python, strings come with a set of built-in **methods** that allow us to manipulate and work with text efficiently. These methods are called using the dot notation, like `string.method()`. **Methods** are functions that are associated with an object (of certain class), and called using dot notation, whereas functions are independent blocks of code that can be called on their own and called using `function(string)`.

```
In [4]: string
```

```
Out[4]: 'Python is fun!'
```

```
In [5]: print(string.upper())
        print(string.lower())
        print(string.title())
```

PYTHON IS FUN!
python is fun!
Python Is Fun!

Strings formatting

In some situations, we would need to use the variable's value inside our string. We can do this in python as follows:

```
In [ ]: Name = 'Samaher'
text_1 = f'My name is {Name}'
print(text_1)
```

```
In [ ]: string
```

```
In [ ]: text_2 = 'I will learn more'
print(string, "\n\t", text_2)
```

```
In [ ]: part_1 = 'Abrar'
part_2 = ' '
part_3 = 'Ahmed'
empty_string = ''
full_text = part_1 + part_2 + part_3
print(full_text)
```

```
In [ ]: full_text = part_1 + empty_string + part_3
print(full_text)
```

String Indexing

In Python, there are two ways to index elements in a sequence such as a string. The first is indexing from the left, which starts at `0` and increases as we move to the right.

0	1	2	3	4	5	6	7	8	9	10
H	e	l	l	o		w	o	r	l	d

The second is indexing from the right, known as negative indexing. It starts at `-1` for the last element and decreases as we move to the left.

H	e	l	l	o		w	o	r	l	d
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

```
In [ ]: string = 'Hello world'
        string[0]
```

```
In [ ]: string[-3]
```

For slicing we follow the rule `string[strat : stop]` , where the strat is included, stop is excluded.

```
In [ ]: string[0:3]
```

```
In [ ]: string[3:6]
```

```
In [ ]: string[-4:-1]
```

```
In [ ]: string[::-1] # This reverse the string
```

```
In [ ]: string[0:3] + string[9:11]
```

```
In [ ]: string[-11:-5] + string[6:]
```

```
In [ ]: # string[start : stop : step]
        string[::2] # every second element
```

```
In [ ]: string[2::2]
```

Numbers

Another fundamental data type in Python is **numbers**. Numbers are used to represent numeric values and perform mathematical calculations. Python mainly works with two common numeric types: integers and floating-point numbers.

```
In [ ]: a = 5
        b = 2.5
        print(type(a), type(b))
```

```
In [ ]: c = a + b
        c
```

```
In [ ]: d = c/b
        d
```

```
In [ ]: f = d ** 2
        f
```

```
In [ ]: speed_of_light = 300_000_000
        speed_of_light
```

```
In [ ]: speed_of_light = 3e8
        speed_of_light
```

```
In [ ]: x, y, z = 20, 0, 0.5
        x, y, z
```

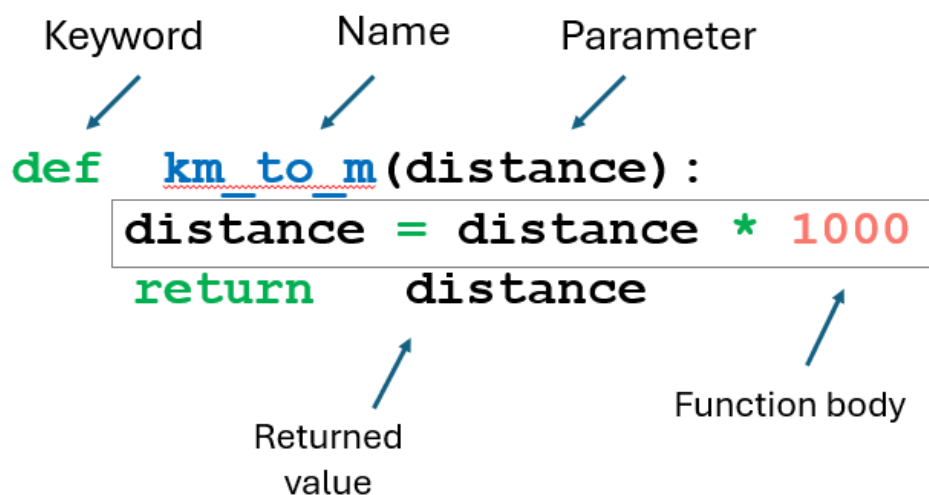
Functions in Python

A function is a block of code that performs a specific task and can be given a name. It enables us to:

- Reuse code
- Abstract functionality
- Divide complex problems into smaller parts(think in smaller chunks)
- Make programs easier to test and maintain.



Here is simple function the convert the distance from kilometer to meter:



- The variable `distance` in the definition of `km_to_m()` is a **parameter**, whereas the value we pass from function call to the function is **argument**, such as the value 2.5 in the call `km_to_m(2.5)`

```
In [ ]: # Function with no parameters.
        def greet_user():
            return "Hello"
```

```
greet_user()
```

```
In [ ]: # Function with one parameters.  
def say_hello(name):  
    return "Hello " + name  
  
say_hello("Asmaa")
```

Positional Argument

```
In [ ]: def user_info(name, age):  
        text = f"My name is {name} and I am {age} years old."  
        return text  
  
user_info('Nora', 25)
```

Order matters in positional argument.

```
In [ ]: user_info(25, 'Nora')
```

Keyword Argument

```
In [ ]: user_info(age= 20, name= 'Sara')
```

Default Value

```
In [ ]: def user_info(name, age= 15):  
        text = f"My name is {name} and I am {age} years old."  
        return text  
  
user_info('Waad')
```

```
In [ ]: print(user_info('Waad', 29))  
print(user_info('Farah', age= 22))
```

When you use default values, make sure that you pass the default parameter after all other non default parameters. This allow Python to continue interpreting positional arguments correctly.

```
In [ ]: print(user_info(age= 22, 'Sumyiah'))
```

Optional Argument

```
In [ ]: def user_info(name, age, city= False):  
        if city:  
            text = f"My name is {name}, I am {age} years old, and I live in {city}."  
        else:  
            text = f"My name is {name}, I am {age} years old."  
        return text  
  
user_info('Nora', 25)
```

```
In [ ]: user_info('Nora', 25, 'Jeddah')
```

Arbitrary Number of Arguments

```
In [ ]: def make_salad(*veggies):  
        """This function prints the list of all ingredients"""  
        print(veggies)  
  
        make_salad('Tomato')  
        make_salad('Tomato', 'Lettuce', 'Cucumber')
```

Usually, you will see the generic parameters named `*args`, which tell you to pass arbitrary positional arguments.

Variables of the Functions.

```
In [ ]: def test_function():  
        test_var = 'Hello'  
        return test_var  
  
        print(test_var)
```

```
In [ ]: global_variable = 3e8  
  
def Energy_from_Mass(mass):  
    """  
    E = mass * c**2, where c is the speed of light constant.  
    """  
    E = mass * global_variable  
    return E  
  
print(Energy_from_Mass(2.24e3))
```

```
In [ ]: global_variable_test = 10  
  
def test_global_var():  
    global_variable_test = 44  
    return global_variable_test  
  
print(global_variable_test)
```

```
In [ ]: books = 4  
  
def test_global_var2():  
    global books  
    books += 1 # or books = books + 1  
    return books  
  
print('Variable inside the function', test_global_var2())  
print('Variable outside the function', books)
```

Note:

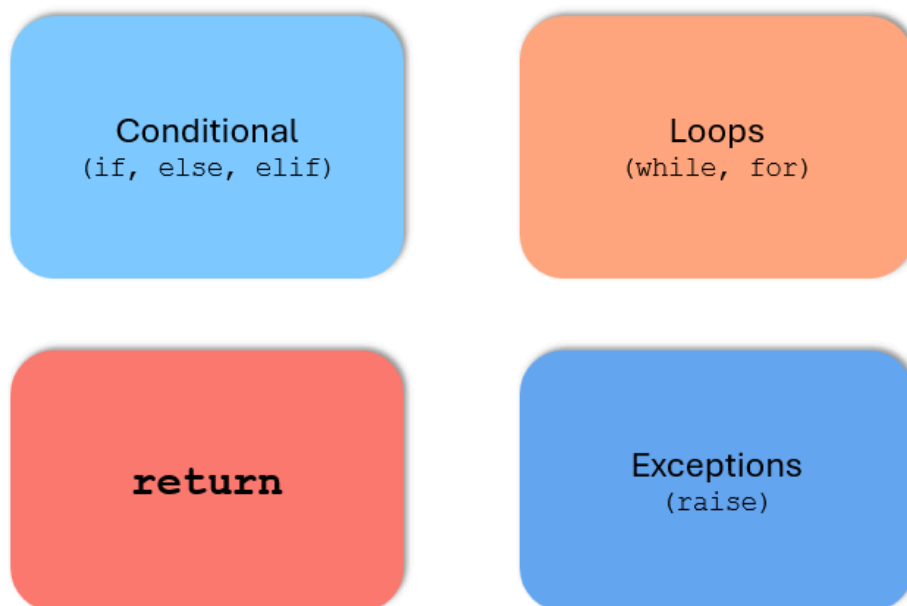
Modifying global variables is generally not recommended because it can make functions unstable and harder to control. In general, relying on global variables is not considered a good programming practice.

Nested Functions.

```
In [ ]: def final_price(price):  
        def tax(price):  
            tax_rate = 0.15  
            return price * tax_rate  
        price += tax(price)  
        return price  
  
final_price(100)
```

Control Flow Structures.

Control flow structure determine how a program excute the instructions such as, make decisions, repeat blocks of code, return values and raise error exceptions.



If Statement

You must already experienced the conditional statement in MATLAB, so that you are familiar with its uses and purposes. In Python, the `if` statement serves the same purpose, it allows a program to make decisions and execute certain code only when a specified condition is true.

The basic syntax of an `if` statement in Python is:


```
if condition:
    statement
```

- No parentheses needed around the condition.
- A colon (:) is essential at the end of the `if` line.
- The alignment is important.
- There is no `end` keyword.

```
In [ ]: x = 6
if x > 10:
    print("x is greater than 10")
elif x > 5:
    print("x is greater than 5 but not greater than 10")
else:
    print("x is 5 or less")
```

```
In [ ]: def calculate(a, b, operation):
    def add():
        return a + b

    def subtract():
        return a - b

    def multiply():
        return a * b

    def divide():
        return a / b

    # For the third argument we need.
    if operation == 'add':
        return add()
    elif operation == 'subtract':
        return subtract()
    elif operation == 'multiply':
        return multiply()
    elif operation == 'divide':
        return divide()

calculate(4, 2, 'divide')
```

```
In [ ]: def is_numeric(number):
    if type(number) == int or type(number) == float:
        return True
    return False

is_numeric('s')
```

```
In [ ]: a = 3.333
if a > 0 and a % 2 == 0:
    print('a is even position integer')
elif type(a) != int:
    print('a is a real number')
```

```
In [ ]:
```

Boolean values are subtypes of integers.

```
In [ ]: print(type(True), type(x))
        print(isinstance(True, type(x)))
        print(isinstance(False, type(x)))
```

```
In [ ]: if True:
        print('This block excuted')
```

```
In [ ]: if 5:
        print('This block excuted')
```

```
In [ ]: if 'string':
        print('This block excuted')
```

Loops

In Python, loops allow us to repeat a block of code multiple times. The two main types of loops are the `for` loop and the `while` loop. The syntax of the loop in Python will be:

```
while [condition]:

    [Body (instructions)]

    [Increment counter / Breaking the loop]
```

- When to use each?

Situation	Use
You know the number of iterations	<code>for</code> loop
You are looping over a sequence	<code>for</code> loop
You loop until a condition changes	<code>while</code> loop
Number of iterations is unknown	<code>while</code> loop

```
In [ ]: counter = 0
        while counter < 5:
            print(counter)
            counter += 1
```

```
In [ ]: for i in range(5):
        print(i)
```

```
In [ ]:
```

```
In [ ]: for i in range(5):
        if i == 2:
```

```
        continue
    print(i)
```

```
In [ ]: for i in range(5):
        if i == 2:
            break
        print(i)
```

Collections

In Python, **collections** are container data types used to store multiple items in a single variable. There are so many different collection and each type has its traits, purpose and way to deal with. Some collection will suit better than other for particular domain. Sometimes, collections are more performant or run faster or save memory storage. Also, it help to orgnize data based on types which ease the maintiance and make the code readable. Python has several built-in collection types:

- List
- Tuple
- Set
- Dictionary

Lists

```
In [ ]: user1 = 'Maram'
        user2 = 'Mariam'
        user3 = 'Samyrah'
        users = [user1, user2, user3]
        users
```

Heterogeneous

All Python collections are heterogeneous. that is, it can store any type of data in one variable.

```
In [ ]: List1 = [True, 3, 'Hi', 3.14]
        List1
```

Note: The ability of mixing strings, integers and boolean does not imply we should do mix them. We are obligated to to keep consistent of data type for meaningful usage.

Ordered

Lists are ordered sequences. That is, they will remember the order (index) that you chose for the elements.

```
In [ ]: print(users) # The order is preserved
```

Mutable

The ability to mutate the lists is available, we can remove, add, or update elements in the list.

```
In [ ]: users.remove('Maram')
        print(users)
```

Indexing and Slicing

Lists and strings share many similarities, since they are both sequences. A list is sequence of objects, while a string is sequence of characters. Therefore, many operations are performed in the same way of sequences.

```
In [ ]: len(users)
```

```
In [ ]: users[1]
```

```
In [ ]: users[1] = 'Sarah'
        print(users)
```

```
In [ ]: len(users[1])
```

```
In [ ]: List2 = ['A', 'B', 'C', 'D', 'E']
        List2[0:2]
```

```
In [ ]: List2[1:]
```

```
In [ ]: List2[-4:-1]
```

Growing Lists

append and insert

- `append` method is used to add element to the end of a list.
- `insert` method is used to add an element of certain location to the list.

```
In [ ]: users
```

```
In [ ]: users.append('Asmaa')
        users
```

```
In [ ]: users.append('May')
        users
```

```
In [ ]: users.insert(0, 'Fatimah')
        users
```

Other Operations

```
In [ ]: users.index('Sarah') # Used to query about location
```

```
In [ ]: 'Mariam' in users # Used to check of existence of an element.
```

```
In [ ]: List3 = [3 , 4, 2.33, 10]
List4 = ['h', 's', 'a', 'c', 'b']
```

```
In [ ]: min(List3), max(List3)
```

```
In [ ]: min(List4), max(List4)
```

```
In [ ]: List3.reverse()
print(List3)
```

```
In [ ]: List3.count(3)
```

```
In [ ]: List5 = List3.copy()
List5
```

```
In [ ]: List5.remove(10)
print(List3)
print(List5)
```

Sorting Elements

For mutable collection we use, `sort()` method. On the other hand, we use `sorted()` built-in function for immutable collections

```
In [ ]: List3.sort()
List3
```

```
In [ ]: sorted(List4)
```

Duplicated

```
In [ ]: list6 = [1 ,1 ,2, 5]
list6
```

```
In [ ]: list6.count(1)
```

Tuples

A tuple is a type of **ordered** sequence similar to lists. But, the difference between them is tuples **immutable**. That means you **CAN NOT** change them once they are created. This is useful for situations where we need to provide safety. Same as lists, tuples allow **duplicated** elements.

Creating Tuple

Creating Tuple is similar to creating a list. But instead of using the square bracket `[]`, we will use the parentheses `()`

```
In [ ]: tuple1 = ('a', 'd', 'b')
        tuple1
```

There is also a `tuple()` function which you could create a tuple from a list

```
In [ ]: tuple2 = tuple(users)
        tuple2
```

We can also create tuple from other tuples. Note that we are constructing new tuple from other existing tuples, not mutating them. That is `tuple1` and `tuple2` are not changed.

```
In [ ]: tuple3 = tuple1 + tuple2
        tuple3
```

```
In [ ]: tuple1 += ('X', 'Y', 'Z')
        tuple1
```

```
In [ ]: print(tuple1, tuple2)
```

Operation

Tuples support most common Python operations.

```
In [ ]: len(tuple1)
```

```
In [ ]: 'a' in tuple3
```

```
In [ ]: min(tuple1), max(tuple1)
```

```
In [ ]: tuple1.count('a')
```

Indexing and Slicing

works in the same way as with lists.

Dictionaries

The biggest weakness of lists is that they do not offer much information about the values they are storing. Previously, we demonstrated how lists are automatically indexed from right or left. But, sometimes we would like an index to be a readable understandable keyword that explains the value we are storing. For example, if I have the two lists of user first

name and user surname are used for some purpose. These two lists offer different information that can easily be mixed or lead to mislead. Therefore, we would like to give a key for each of those lists to distinguish them. This can be done by dictionaries.

Dictionaries are used to store data values in key:value pairs. A dictionary is a collection which is **unordered**, **mutable** and **DO NOT allow duplicates**.

```
In [ ]: user_first_name = users
        user_first_name
```

```
In [ ]: user_last_name = ['Al-frra', 'Ahmed', 'Al-Ahmari', 'Al-ghamdi', 'Al-mossa']
```

```
In [ ]: user_city = ['Jeddah', 'Dammam', 'Abha', 'Albaha', 'Jeddah']
```

```
In [ ]: users = [user_first_name, user_last_name, user_city]
        users[0] # we do not know what this list refer to
```

```
In [ ]: users[user_first_name]
```

Instead, we can create a dictionary that save the value's key.

```
In [ ]: user = {
        'First_Name' : 'Sarah',
        'Last_Name' : 'Al-Ahmari',
        'city' : 'Abha'
    }
    user
```

```
In [ ]: user['First_Name']
```

Operation on Dictionaries

```
In [ ]: # To add one pair element (key: value)
        user['Age'] = 15
        user
```

```
In [ ]: # To add pairs of elements
        user.update({
            'Email' : 'user1@gmail.com',
            'Gender' : 'Female'
        })
        user
```

```
In [ ]: # To update values.
        user['Age'] = 30
        user
```

```
In [ ]: # Use del to delete element.
        del user['Email']
        user
```

```
In [ ]: # .pop() method also delete an element (recommended).
        user.pop('Gender')
```

```
user
```

```
In [ ]: # Check the existence of key.  
        'First_Name' in user
```

```
In [ ]: # To access keys only.  
        user.keys()
```

```
In [ ]: # To access values only.  
        user.values()
```

```
In [ ]: # To obtain list of key, value pairs (good for iterating)  
        user.items()
```

The dictionary `user` now is constructed to save one user data, to save all users data we defined earlier we will do the following.

```
In [ ]: user.pop('Age')
```

```
In [ ]: user
```

```
In [ ]: user['First_Name'] = user_first_name  
        user['Last_Name'] = user_last_name  
        user['city'] = user_city  
        user
```

This is how we save multiple of users data in one dictionary categorized by keys

Sets

Sets are **Unordered**, **mutable** and efficient type of collection, that has the same few characteristics of sets in set theory such as the **disability to have duplicated items**. Also, sets in Python provide the most popular operation in it. However, sets are one of the less used data structures in Python, it is not needed as dictionaries, tuples and lists.

There are two ways to create sets in Python.

```
In [ ]: set1 = {1, 2, 5}  
        print(set1)  
        print(type(set1))
```

```
In [ ]: set2 = set(['red', 'green', 'yellow'])  
        print(set2)  
        print(type(set2))
```

Operation on Sets

```
In [ ]: # To add one element.  
        set2.add('Tale')
```



```
set2
```

```
In [ ]: # To add multiple elements.
        set1.update({3, 6})
        set1
```

We do not know which element will be deleted using `.pop()` , since sets are unordered.

```
In [ ]: # Delete an element.
        set2.pop()
        set2
```

```
In [ ]: # Specify the deleted element.
        set2.remove('red')
        set2
```

Now, we will see how to perform the set mathematical operation.

```
In [ ]: # Union
        set1 | set2
```

```
In [ ]: # Intersection
        set1 & set2
```

```
In [ ]: set3 = {3, 2, 11}
```

```
In [ ]: # Difference
        set1 - set3
```

```
In [ ]: # Subset Checking
        set1 > set3
```

Quick Comparison

Type	Ordered	Mutable	Duplicates	Use Case
List	✓	✓	✓	General purpose
Tuple	✓	✗	✓	Fixed data
Set	✗	✓	✗	Unique items
Dictionary	✓	✓	Keys ✗	Key-value data

Iterating over List

```
In [ ]: # Befor we did
        for i in range(5):
            print(i) # Do somethin five times
```

```
In [ ]: list6 = [1, 8, 9, 10, 13]
        for i in list6:
            print(i)
```

```
In [ ]: largest_num = None
        for i in list6:
            if largest_num is None or i > largest_num:
                largest_num = i
        print(largest_num)
```

```
In [ ]: list7 = ['Orange', 'Apple', 'Banana']
        list(enumerate(list7))
```

```
In [ ]: for index, fruit in enumerate(list7):
        print(fr"{fruit} is in position: {index}")
```

Iterating Over Dictionary

```
In [ ]: user
```

```
In [ ]: user.items()
```

```
In [ ]: key_num = 1
        for key, value in user.items():
            print(fr'key {key_num} is {key}, its values is {value}')
            key_num +=1
```

Object-Oriented Programming

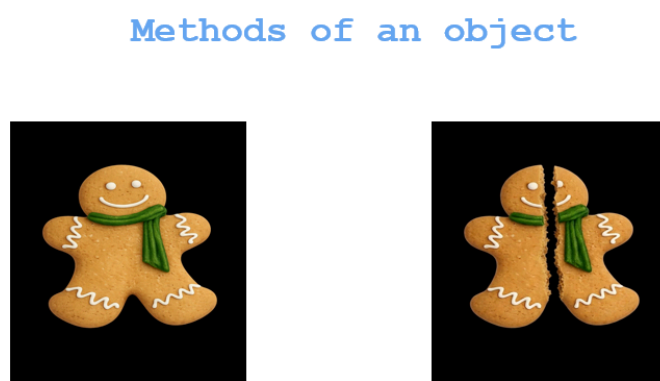
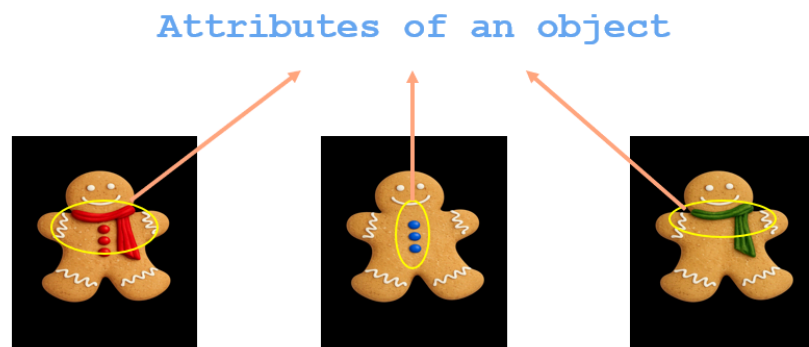
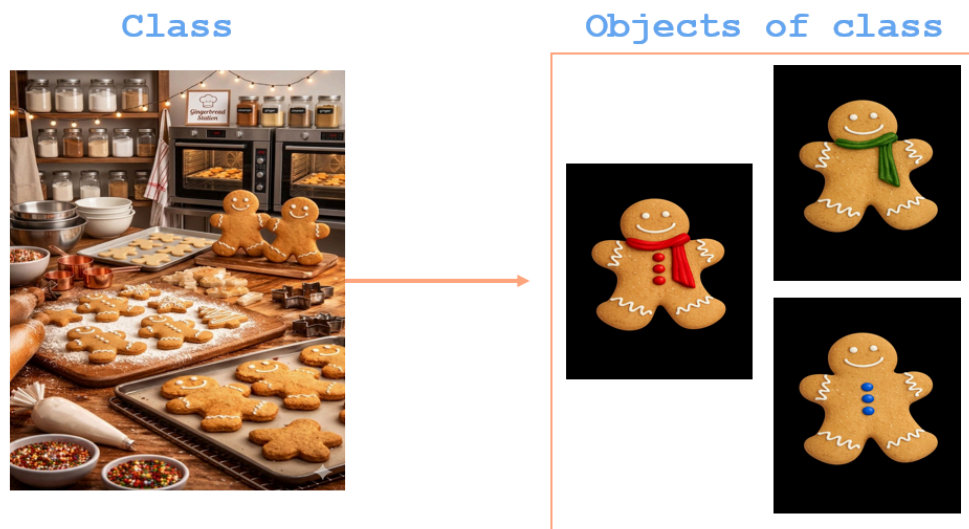
Object-Oriented Programming (OOP) is a programming paradigm that organizes code around **objects** and **classes** rather than functions and logic alone. It focuses on creating **reusable, modular**, and scalable code by modeling real-world entities.

In OOP, data (**attributes**) and behavior (**methods**) are bundled together into classes, which act as blueprints for creating objects. These objects interact with each other to perform various tasks, similar to how real-world entities interact.

Why we need OOP?

- Helps you **reuse code** using concepts like **inheritance** (creating new classes from existing ones).
- Makes code easier to understand by organizing it into **small, focused parts**.
- Simplifies fixing bugs and updating programs.
- Keeps data safe by controlling how it's accessed.
- Allows flexible code where objects can behave in different ways.

Simple Analogy of Classes



Classes and Objects

A class is a blueprint or template that defines the structure and behaviour (attributes and methods) of objects. An object is an instance of a class that holds actual data. Classes are used to represent real-world entities like cars, users, or bank accounts in

code. Objects created from these classes carry unique data while sharing the same structure and functionality.

This is how we construct simple class in Python:

```
In [ ]: class Cookie(object):  
        pass
```

```
In [ ]: cookie_1 = Cookie()  
        cookie_2 = Cookie()
```

`cookie_1` and `cookie_2` are objects (entities) of the class `Cookie()`. Each object is unique individual with unique memory address.

```
In [ ]: cookie_1
```

```
In [ ]: cookie_2
```

Arrtibutes of Class

For this example, we assume that its traits are the buttons and scarfs.

```
In [ ]: cookie_1.buttons = 'blue' # we gave it blue buttons  
        cookie_1.buttons
```

```
In [ ]: cookie_2.scarf = 'red'  
        cookie_2.scarf
```

```
In [ ]: cookie_2.flavor
```

Methods of Class

```
In [ ]: class Cookie(object):  
        default_scarf_color = 'red'
```

It behaves just like an object's attribute when you access it, but the key difference is that it belongs to the class rather than to each individual object. This means it is shared across all instances of the class, instead of being unique to a single object.

```
In [ ]: Cookie.default_scarf_color
```

```
In [ ]: Cookie.default_scarf_color = 'green'  
        Cookie.default_scarf_color
```

Objects can access class attributes also from the method.

```
In [ ]: class Cookie(object):  
        defalut_scarf_color = 'green'  
        def __init__(self, buttons, scarf = None):  
            # Scarf is Optional  
            self.buttons = buttons  
            if scarf:
```

```

        self.scarf = scarf
    else:
        # Access the class default attribute
        self.scarf = self.default_scarf_color

```

```

In [ ]: cookie_1 = Cookie('black')
        cookie_2 = Cookie('tale', 'Pink')
        cookie_3 = Cookie(buttons= 'yellow', scarf= 'white')

```

```

In [ ]: print(cookie_1.buttons, ', ', cookie_2.buttons, ', ', cookie_3.buttons)

```

```

In [ ]: print(cookie_1.scarf, ', ', cookie_2.scarf, ', ', cookie_3.scarf)

```

```

In [ ]: class Cookie(object):
        default_scarf_color = 'green'
        default_cookie_weight = 100

        def __init__(self, buttons, scarf=None, weight = None):
            self.buttons = buttons
            if scarf:
                self.scarf = scarf
            else:
                self.scarf = self.default_scarf_color
            if weight:
                self.weight = weight
            else:
                self.weight = self.default_cookie_weight

        def chop_in_half(self):
            self.weight = self.weight / 2
            return "The cookie has been chopped in half!"

```

```

In [ ]: cookie_1 = Cookie('black')
        cookie_2 = Cookie('tale', 'Pink', 300)
        cookie_3 = Cookie(buttons= 'yellow', scarf= 'white')

```

```

In [ ]: cookie_1.weight

```

```

In [ ]: cookie_2.weight

```

```

In [ ]: cookie_2.chop_in_half()

```

```

In [ ]: cookie_2.weight

```